1️⃣ View vs Copy

Term	Description
View	A new array object that **shares the same memory** as original. Changes affect original.
Copy	A new array object with **separate memory**. Changes **do NOT** affect original.

2️⃣ How to create a View

```
arr = np.array([1,2,3,4])

v = arr.view()

v[0] = 100

print(arr)  # [100,2,3,4] → original is affected
```

- .view() creates shallow copy → shared data

3️⃣ How to create a Copy

```
c = arr.copy()

c[1] = 200

print(arr)  # [100,2,3,4] → original unchanged
```

- .copy() creates deep copy → independent data

4️⃣ Slicing Behavior

```
arr = np.array([1,2,3,4,5])

s = arr[1:4]

s[0] = 99

print(arr)  # [1,99,3,4,5] → slice is **view**, original affected
```

- **Important**: slicing returns a **view**, not copy

5️⃣ Reshape Behavior

```
arr = np.arange(6)
```

r = arr.reshape(2,3)

r[0,0] = 100

print(arr) # [100,1,2,3,4,5] → reshape returns view if possible

- reshape returns **view** when memory is contiguous
- Otherwise, returns copy

6️⃣ When to Use Copy vs View

Scenario	Use
Want to modify array **without changing original**	.copy()
Temporary reshape, slice, or broadcast	.view() or slicing
Performance critical, memory saving	.view()

7️⃣ How to Check If Two Arrays Share Memory

arr = np.arange(5)

v = arr.view()

c = arr.copy()

np.shares_memory(arr, v) # True

np.shares_memory(arr, c) # False

8️⃣ Mini Rules Summary

1. **Slice → view**
2. **reshape → view if contiguous**
3. **.copy() → always independent**
4. **.view() → shared memory**
5. **Fancy indexing / boolean indexing → always copy**

⚠ This is why:

arr[[0,2]] # fancy indexing → copy

```
arr[arr>2]  # boolean indexing → copy
```